

**LAPORAN PRAKTIKUM  
ALGORITMA & STRUKTUR DATA  
MODUL 6**



**Graph dan Tree**

**Oleh:**

**Nabilah Nur Azizah     NIM. 2510817120015**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
JUNI 2026**

**LEMBAR PENGESAHAN**  
**LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA**  
**MODUL 6**

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nabilah Nur Azizah  
NIM : 2510817120015

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani  
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D  
NIP. 198606132015041001

## DAFTAR ISI

|                         |     |
|-------------------------|-----|
| LEMBAR PENGESAHAN ..... | i   |
| DAFTAR ISI.....         | ii  |
| DAFTAR GAMBAR .....     | iii |
| DAFTAR TABEL .....      | iv  |
| SOAL 1 .....            | 1   |
| A. Source Code .....    | 1   |
| B. Output Program ..... | 4   |
| C. Pembahasan .....     | 4   |
| SOAL 2 .....            | 8   |
| A. Source Code .....    | 8   |
| B. Output Program ..... | 11  |
| C. Pembahasan .....     | 11  |
| SOAL 3 .....            | 14  |
| A. Source Code .....    | 15  |
| B. Output Program ..... | 17  |
| C. Pembahasan .....     | 17  |
| SOAL 4 .....            | 21  |
| A. Source Code .....    | 21  |
| B. Output Program ..... | 24  |
| C. Pembahasan .....     | 24  |
| SOAL 5 .....            | 27  |
| A. Source Code .....    | 27  |
| B. Output Program ..... | 31  |
| C. Pembahasan .....     | 31  |

## **DAFTAR GAMBAR**

|                                                 |    |
|-------------------------------------------------|----|
| Gambar 1. Screenshot Hasil Jawaban Soal 1 ..... | 4  |
| Gambar 2. Screenshot Hasil Jawaban Soal 2 ..... | 11 |
| Gambar 3. Screenshot Hasil Jawaban Soal 3 ..... | 17 |
| Gambar 4. Screenshot Hasil Jawaban Soal 4 ..... | 24 |
| Gambar 5. Screenshot Hasil Jawaban Soal 5 ..... | 31 |

## DAFTAR TABEL

|                                   |    |
|-----------------------------------|----|
| Tabel 1. Source Code Soal 1 ..... | 1  |
| Tabel 2. Source Code Soal 2 ..... | 8  |
| Tabel 3. Source Code Soal 3 ..... | 15 |
| Tabel 4. Source Code Soal 4 ..... | 21 |
| Tabel 5. Source Code Soal 5 ..... | 27 |

## SOAL 1

Diberikan sebuah directed **weighted graph** yang terdiri dari  $N$  vertex dan  $M$  edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai  $U, V, W$ , yang berarti terdapat edge berarah dari vertex  $U$  menuju vertex  $V$  dengan bobot  $W$ . Karena graph bersifat berarah, edge  $U, V, W$  tidak otomatis berarti terdapat edge dari  $V$  menuju  $U$ .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- $i$  menuju vertex ke- $j$ , maka nilai matrix pada baris  $i$  dan kolom  $j$  adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

| Input                                                                     | Output                                                                                                    |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 5<br>A B C D E<br>6<br>A B 4<br>A C 2<br>B D 7<br>C B 1<br>C E 6<br>D C 3 | Adjacency Matrix:<br>A B C D E<br>A 0 4 2 0 0<br>B 0 0 0 7 0<br>C 0 1 0 0 6<br>D 0 0 3 0 0<br>E 0 0 0 0 0 |

### A. Source Code

Tabel 1. Source Code Soal 1

|   |                      |
|---|----------------------|
| 1 | #include <iostream>  |
| 2 | #include <vector>    |
| 3 |                      |
| 4 | using namespace std; |

```

5
6 int main() {
7     int N;
8     if (!(cin >> N)) return 0;
9
10    vector<char> vertices(N);
11    int charToIndex[256];
12    for (int i = 0; i < 256; i++) {
13        charToIndex[i] = -1;
14    }
15
16    for (int i = 0; i < N; i++) {
17        cin >> vertices[i];
18        charToIndex[vertices[i]] = i;
19    }
20
21    int M;
22    cin >> M;
23
24    vector<vector<int>> adjMatrix(N, vector<int>(N,
25    0));
26
27    for (int i = 0; i < M; i++) {
28        char u, v;
29        int w;
30        cin >> u >> v >> w;
31
32        int u_idx = charToIndex[u];
33        int v_idx = charToIndex[v];

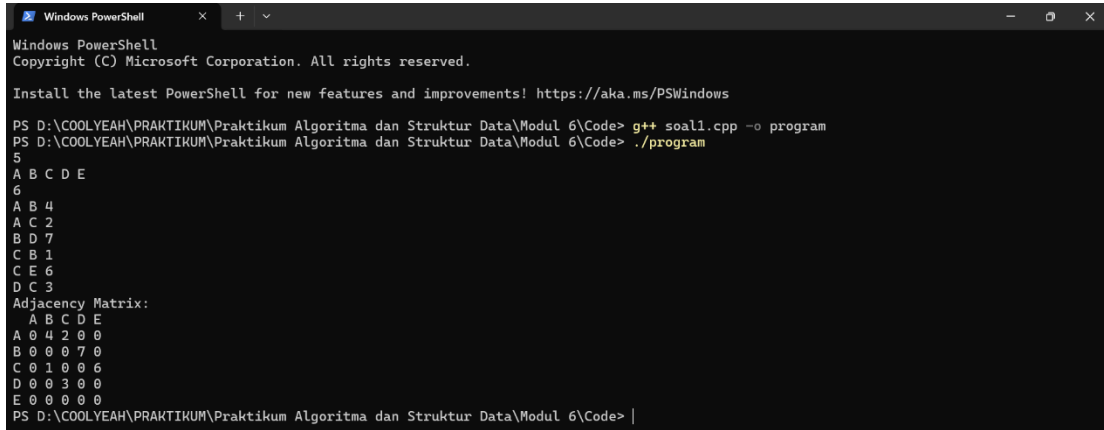
```

```

34         if (u_idx != -1 && v_idx != -1) {
35             adjMatrix[u_idx][v_idx] = w;
36         }
37     }
38
39     cout << "Adjacency Matrix:\n";
40     cout << " ";
41     for (int i = 0; i < N; i++) {
42         cout << " " << vertices[i];
43     }
44     cout << "\n";
45
46     for (int i = 0; i < N; i++) {
47         cout << vertices[i];
48         for (int j = 0; j < N; j++) {
49             cout << " " << adjMatrix[i][j];
50         }
51         cout << "\n";
52     }
53
54     return 0;
55 }

```

## B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> g++ soal1.cpp -o program
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> ./program
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> |
```

Gambar 1. Screenshot Hasil Jawaban Soal 1

## C. Pembahasan

Kode ini bertujuan untuk mengubah format data graf dari bentuk daftar sisi (*Edge List*) menjadi matriks ketetanggaan (*Adjacency Matrix*). Graf yang dibangun adalah jenis Graf Berarah dan Berbobot (*Directed & Weighted Graph*). Program akan membaca daftar simpul (*vertex*), membaca daftar sisi koneksi (*edges*) dan bobotnya (*weights*), lalu menyimpan dan menampilkan koneksi tersebut dalam bentuk format matriks 2 dimensi. Program akan memetakan nama simpul (berupa karakter abjad) ke dalam indeks numerik, lalu membangun sebuah *Adjacency Matrix* berukuran  $N \times N$  (di mana  $N$  adalah jumlah simpul). Matriks ini kemudian dicetak ke layar untuk memvisualisasikan dengan jelas simpul mana saja yang saling terhubung beserta nilai bobot koneksinya.

Blok kode pertama berfungsi sebagai *import library* dan persiapan lingkungan. Blok ini bekerja dengan memanggil *library* standar C++. `<iostream>` digunakan untuk menangani input/output data (seperti `cin` dan `cout`), sedangkan `<vector>` digunakan untuk membuat *Dynamic Array* (array yang ukurannya bisa diatur secara dinamis saat program berjalan).

Blok kode kedua berfungsi untuk membaca jumlah simpul dan validasi input. Blok ini diawali dengan membaca jumlah simpul (*vertex*) pada graf yang disimpan dalam variabel *integer* *N*, sekaligus melakukan validasi input menggunakan perintah `if (!(cin >> N)) return 0;`. Mekanisme pengaman ini memastikan bahwa jika input gagal dibaca, misalnya input dibiarkan kosong atau bukan berupa angka, maka program akan langsung berhenti secara aman. Setelah jumlah simpul berhasil divalidasi, fungsi sistem dilanjutkan untuk mengenali dan mendaftarkan identitas setiap simpul.

Mengingat komputer lebih efisien dalam memproses koordinat angka dibandingkan dengan karakter huruf, blok ini membangun sebuah jembatan penerjemah berupa *Lookup Table* atau tabel pemetaan. Cara kerjanya, saat huruf simpul diinput, program tidak sekadar menyimpannya ke dalam daftar utama, tetapi langsung mencatat posisi (indeks) huruf tersebut ke dalam sebuah memori sementara berkapasitas 256 slot yang mewakili seluruh karakter ASCII. Dengan pendekatan ini, ketika program kelak membutuhkan letak suatu huruf tertentu (misalnya huruf 'C'), sistem tidak perlu melakukan pencarian satu per satu secara lambat, melainkan dapat langsung menemukan indeks angkanya seketika.

Blok kode ketiga berfungsi untuk menginisialisasi simpul dan tabel pemetaan, serta menyiapkan ruang kerja matriks. Setelah program mengetahui jumlah simpul yang ada, sistem akan menyiapkan sebuah grid atau tabel dua dimensi berukuran  $N \times N$  (baris kali kolom). Pada tahap awal, seluruh sel di dalam tabel ini diisi dengan angka nol (0) untuk melambangkan kondisi netral atau belum ada jalur koneksi apa pun di antara titik-titik tersebut. Di samping itu, program mendaftarkan simpul dengan mendeklarasikan `vector<char> vertices(N)` untuk menyimpan nama-nama simpul berupa karakter (misalnya 'A', 'B', atau 'C'), serta membuat array statis `int charToIndex[256]` yang bertindak sebagai Tabel Pemetaan (*Lookup Table* atau *Hash Map* sederhana), di mana angka 256 mewakili kapasitas maksimal karakter ASCII.

Untuk mengatur tabel pemetaan ini, perulangan `for` pertama akan mengisi semua elemen `charToIndex` dengan nilai *default* -1, menandakan bahwa karakter tersebut belum dipetakan ke indeks mana pun. Selanjutnya, perulangan `for` kedua bertugas membaca input karakter simpul satu per satu. Saat sebuah karakter dibaca dan disimpan ke dalam `vertices[i]`, program secara otomatis memetakan karakter tersebut ke indeks `i` (*zero-based indexing*) di dalam `charToIndex`. Sebagai contoh, jika simpul pertama yang diinputkan adalah 'A', maka posisinya akan langsung direkam sebagai `charToIndex['A'] = 0`.

Blok kode keempat berfungsi sebagai pemrosesan jalur dan bobot (routing). Proses ini diawali dengan membaca jumlah sisi koneksi (edges) antar simpul yang disimpan ke dalam variabel `M`. Bersamaan dengan itu, program menginisialisasi Adjacency Matrix melalui perintah `vector<vector<int>> adjMatrix(...)` berupa vector 2 dimensi berukuran `N x N`, di mana seluruh sel awalnya disetel ke angka 0 sebagai tanda bahwa secara default belum ada koneksi antar simpul. Selanjutnya, program melakukan perulangan sebanyak `M` kali untuk memproses setiap rute koneksi. Pada setiap iterasi, program meminta tiga informasi utama, yaitu `u` (simpul asal), `v` (simpul tujuan), dan `w` (bobot/weight). Karena matriks beroperasi menggunakan angka, program kemudian menerjemahkan huruf titik asal dan tujuan tersebut menjadi indeks numerik menggunakan *Lookup Table* `charToIndex` yang telah dibuat pada blok sebelumnya.

Melalui blok pengkondisian `if (u_idx != -1 && v_idx != -1)`, sistem memastikan bahwa hanya simpul yang valid dan sudah dipetakan yang akan diproses. Setelah koordinat angkanya didapatkan, program akan menuju sel spesifik di dalam tabel matriks dan menjalankan perintah `adjMatrix[u_idx][v_idx] = w;` untuk menimpa angka nol dengan nilai jarak/bobot yang baru. Proses penempatan jalur yang secara spesifik mengisi baris asal dan kolom tujuan ini menegaskan bahwa graf yang dibangun bersifat Berarah (*Directed*), di mana koneksi hanya berlaku searah dari titik asal ke titik tujuan dan tidak diisi secara bolak-balik (simetris).

Blok kode kelima berfungsi sebagai tahap akhir untuk memformat dan mencetak *Adjacency Matrix* ke layar. Melalui tahap ini, data matriks yang tadinya hanya berupa memori angka di dalam sistem diterjemahkan menjadi tampilan visual yang rapi agar mudah dibaca oleh manusia. Proses pencetakan dimulai dengan pembuatan *header* kolom, perulangan pertama akan mencetak spasi dan menderetkan nama-nama simpul mendatar di baris paling atas sebagai referensi kolom tujuan (*destination*). Selanjutnya, program menggunakan perulangan bersarang (*nested loop*) untuk mencetak isi matriks. Langkah pertama di dalam perulangan luar adalah mencetak `vertices[i]` di bagian paling kiri matriks sebagai judul baris yang mewakili simpul asal (*origin*). Kemudian, perulangan dalam (*inner loop*) akan mencetak deretan nilai koneksi atau bobot yang diambil dari `adjMatrix[i][j]` tepat di sebelahnya. Hasil akhirnya adalah sebuah tabel silang (*cross-tabulation*) yang memperlihatkan dengan jelas titik mana saja yang saling terhubung beserta jaraknya. Setelah seluruh matriks berhasil ditampilkan, perintah `return 0;` dieksekusi untuk menandakan bahwa seluruh rangkaian program `main` telah selesai berjalan dengan sukses.

Kompleksitas waktu keseluruhan dari program ini adalah  $O(N^2)$ . Program ini menjalankan beberapa operasi, yaitu pembuatan *Lookup Table* yang memakan waktu konstan  $O(1)$ , registrasi input simpul memakan waktu linier  $O(N)$ , dan pembacaan jalur koneksi memakan waktu  $O(M)$ . Namun, operasi yang paling mendominasi beban komputasi adalah pembuatan ruang *Adjacency Matrix* 2 dimensi dan proses pencetakannya ke layar. Kedua proses ini menggunakan perulangan bersarang (*nested loop*) berukuran  $N \times N$ , sehingga masing-masing membutuhkan waktu kuadratik sebesar  $O(N^2)$ .

Berdasarkan aturan notasi Big-O yang hanya mengambil faktor pertumbuhan terbesar, total komputasi  $O(N^2 + M + N + 1)$  disederhanakan menjadi  $O(N^2)$ . Kesimpulannya, efisiensi eksekusi program ini sangat dipengaruhi oleh kuadrat dari jumlah simpul yang ada, sesuai dengan karakteristik bawaan dari struktur data *Adjacency Matrix*.

## SOAL 2

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran  $N \times N$ . Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris  $i$  dan kolom  $j$  menunjukkan bobot edge dari vertex ke- $i$  menuju vertex ke- $j$ . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- $i$  menuju vertex ke- $j$ .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input

.

| Input                                                                           | Output                                                                                                    |
|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 5<br>A B C D E<br>0 4 2 0 0<br>0 0 0 7 0<br>0 1 0 0 6<br>0 0 3 0 0<br>0 0 0 0 0 | Adjacency Matrix:<br>A B C D E<br>A 0 4 2 0 0<br>B 0 0 0 7 0<br>C 0 1 0 0 6<br>D 0 0 3 0 0<br>E 0 0 0 0 0 |

### A. Source Code

Tabel 2. Source Code Soal 2

|   |                      |
|---|----------------------|
| 1 | #include <iostream>  |
| 2 | #include <vector>    |
| 3 |                      |
| 4 | using namespace std; |

```

5
6 int main() {
7     int N;
8     if (!(cin >> N)) return 0;
9
10    vector<char> vertices(N);
11    bool isValid[256] = {false};
12
13    for (int i = 0; i < N; i++) {
14        cin >> vertices[i];
15        isValid[vertices[i]] = true;
16    }
17
18    vector<vector<int>> adjMatrix(N, vector<int>(N,
19    0));
20    for (int i = 0; i < N; i++) {
21        for (int j = 0; j < N; j++) {
22            cin >> adjMatrix[i][j];
23        }
24    }
25
26    cout << "Adjacency List:\n";
27    for (int i = 0; i < N; i++) {
28        cout << vertices[i] << " -> ";
29
30        bool hasEdge = false;
31        bool isFirstEdge = true;
32
33        for (int j = 0; j < N; j++) {
34            int weight = adjMatrix[i][j];

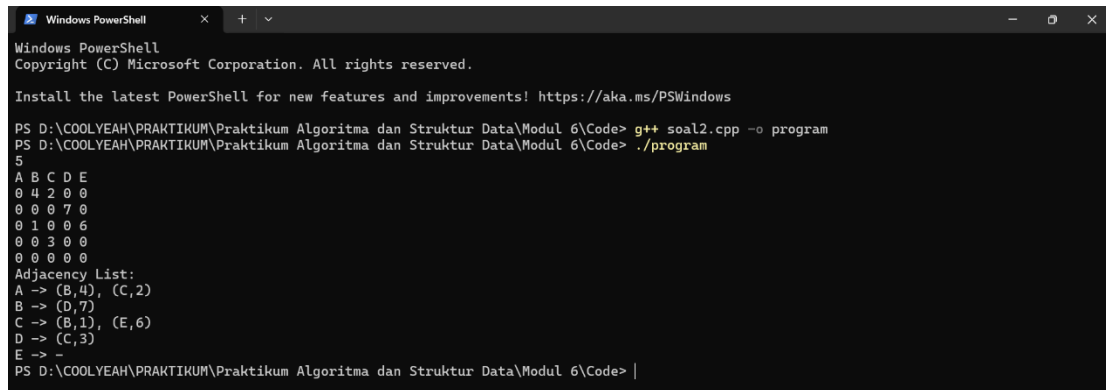
```

```

34
35         if (weight > 0) {
36             // Validasi array boolean
37             if (isValid[vertices[i]] &&
38 isValid[vertices[j]]) {
39                 if (!isFirstEdge) {
40                     cout << ", ";
41                 }
42                 cout << "(" << vertices[j] <<
43                 ", " << weight << ")";
44
45                 hasEdge = true;
46                 isFirstEdge = false;
47             }
48         }
49
50         if (!hasEdge) {
51             cout << "-";
52         }
53         cout << "\n";
54     }
55     return 0;
56 }

```

## B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> g++ soal2.cpp -o program
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> ./program
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> |
```

Gambar 2. Screenshot Hasil Jawaban Soal 2

## C. Pembahasan

Kode ini bertujuan untuk mengubah representasi struktur data graf dari bentuk Adjacency Matrix menjadi Adjacency List. Program akan membaca daftar simpul (karakter), memvalidasi keberadaannya menggunakan array *boolean*, lalu membaca relasi koneksi antar-simpul dari sebuah *Adjacency Matrix* berukuran  $N \times N$ . Data mentah dari matriks tersebut kemudian disaring, dikonversi, dan dicetak ke layar dalam format *Adjacency List* yang secara visual merinci rute setiap simpul beserta bobot tujuannya dengan format tata baca yang rapi.

Blok kode pertama bertindak sebagai persiapan lingkungan kerja program dengan memanggil pustaka standar C++. `<iostream>` digunakan untuk menangani arus input dan output data dasar. Sementara itu, `<vector>` diimpor agar program dapat membuat *Dynamic Array* yang sangat krusial karena jumlah simpul pada graf akan ditentukan secara fleksibel oleh pengguna saat program berjalan (*runtime*), bukan dipatok secara statis di awal.

Blok kode kedua berfungsi untuk mengelola registrasi simpul sekaligus membangun sistem keamanan validasi data. Dimulai dengan membaca jumlah total simpul ( $N$ ) dengan sebuah validasi anti-gagal, jika input kosong atau bukan angka

maka program secara otomatis berhenti dengan aman. Setelah itu, program menyiapkan *vector* karakter untuk menampung nama-nama simpul dan sebuah array statis *isValid* berkapasitas 256 slot (mewakili seluruh karakter ASCII) dengan nilai awal *false*. Saat pengguna menginput karakter simpul satu per satu, program menyimpannya sekaligus menandai indeks karakter tersebut menjadi *true* di dalam array *isValid*. Logika tabel pemetaan ini memastikan bahwa pada proses selanjutnya, hanya karakter yang benar-benar terdaftar di awal yang akan diakui dan diproses oleh sistem.

Blok ketiga berfokus pada penerimaan dan penyimpanan data koneksi graf. Sistem menyiapkan ruang memori berupa matriks dua dimensi berukuran  $N \times N$  menggunakan tipe data *vector*, dengan nilai seluruh sel diatur ke angka 0 secara *default*. Melalui perulangan *nested loop*, program secara sistematis membaca nilai dari input pengguna dan mengisikannya baris demi baris ke dalam matriks. Matriks ini bertindak sebagai memori penyimpanan bobot (*weight*) mentah yang merekam koneksi antar simpul, di mana nilai di atas angka 0 menandakan adanya jalur aktif beserta jaraknya, sedangkan angka 0 berarti tidak ada koneksi.

Blok keempat adalah inti pemrosesan program di mana matriks angka dikonversi menjadi *Adjacency List* yang lebih informatif secara visual. Program mengevaluasi setiap baris dari simpul asal dengan memindai seluruh kolom simpul tujuan. Saat menemukan angka bobot lebih dari 0, sistem melakukan verifikasi ganda memanggil *isValid* untuk memastikan bahwa simpul asal dan tujuannya benar-benar terdaftar. Jika berhasil terverifikasi, jalur koneksi akan dicetak ke layar dengan format `(Tujuan,Bobot)`. Selain itu, blok ini juga menerapkan logika presentasi khusus menggunakan dua variabel penanda (*flag*), yaitu *isFirstEdge* untuk memastikan pemisah koma disematkan dengan benar di antara rute tanpa berlebih di awal atau akhir rute, dan *hasEdge* yang bertugas mendeteksi jika suatu simpul berstatus buntu atau tidak memiliki rute keluar), sehingga program dapat mencetak tanda hubung (-) sebagai indikator kekosongan. Proses diakhiri dengan `return 0;`, menandakan seluruh konversi sukses dijalankan tanpa kendala.

Kompleksitas waktu keseluruhan dari program ini adalah  $O(N^2)$ . Program ini menjalankan beberapa tahapan operasi, yaitu inisialisasi array validasi *boolean* dan pembacaan daftar simpul awal yang memakan waktu linier sebesar  $O(N)$ . Selanjutnya, operasi yang paling mendominasi beban komputasi adalah tahap pengisian data *Adjacency Matrix* dari input dan tahap pencetakannya ke format *Adjacency List*. Kedua proses tersebut sama-sama menggunakan perulangan *nested loop* berukuran  $N \times N$ , sehingga masing-masing membutuhkan waktu komputasi kuadratik sebesar  $O(N^2)$ . Pengecekan kondisi di dalam perulangan tersebut (seperti memvalidasi `weight > 0` dan mengakses array `isValid`) berjalan sangat cepat dalam waktu konstan  $O(1)$ .

Berdasarkan aturan asimtotik notasi Big-O yang mengabaikan konstanta dan hanya mengambil faktor pertumbuhan dominan, total komputasi  $O(2N^2 + N + 1)$  dapat disederhanakan menjadi  $O(N^2)$ . Hal ini menunjukkan bahwa efisiensi eksekusi program ini sangat bergantung pada luas dimensi matriks yang diproses, di mana waktu komputasinya akan bertumbuh secara kuadratik seiring dengan penambahan jumlah simpul.

### SOAL 3

Sebuah labirin direpresentasikan sebagai grid berukuran  $R \times C$ . Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

| Input                                                                                                                                                                                                      | Output |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| 8 10<br>0 0 0 0 1 0 0 0 0 0<br>1 1 1 0 1 0 1 1 1 0<br>0 0 1 0 0 0 0 0 1 0<br>0 1 1 1 1 1 1 0 1 0<br>0 0 0 0 0 0 1 0 0 0<br>1 1 1 1 1 0 1 1 1 0<br>0 0 0 0 1 0 0 0 1 0<br>0 1 1 0 0 0 1 0 0 0<br>0 0<br>7 9 | 16     |

## A. Source Code

*Tabel 3. Source Code Soal 3*

|    |                                                                                    |
|----|------------------------------------------------------------------------------------|
| 1  | <code>#include &lt;iostream&gt;</code>                                             |
| 2  | <code>#include &lt;vector&gt;</code>                                               |
| 3  | <code>#include &lt;queue&gt;</code>                                                |
| 4  |                                                                                    |
| 5  | <code>using namespace std;</code>                                                  |
| 6  |                                                                                    |
| 7  | <code>struct State {</code>                                                        |
| 8  | <code>    int r, c, dist;</code>                                                   |
| 9  | <code>};</code>                                                                    |
| 10 |                                                                                    |
| 11 | <code>int main() {</code>                                                          |
| 12 | <code>    int R, C;</code>                                                         |
| 13 | <code>    if (!(cin &gt;&gt; R &gt;&gt; C)) return 0;</code>                       |
| 14 |                                                                                    |
| 15 | <code>    vector&lt;vector&lt;int&gt;&gt; grid(R, vector&lt;int&gt;(C));</code>    |
| 16 | <code>    for (int i = 0; i &lt; R; i++) {</code>                                  |
| 17 | <code>        for (int j = 0; j &lt; C; j++) {</code>                              |
| 18 | <code>            cin &gt;&gt; grid[i][j];</code>                                  |
| 19 | <code>        }</code>                                                             |
| 20 | <code>    }</code>                                                                 |
| 21 |                                                                                    |
| 22 | <code>    int SR, SC, FR, FC;</code>                                               |
| 23 | <code>    cin &gt;&gt; SR &gt;&gt; SC &gt;&gt; FR &gt;&gt; FC;</code>              |
| 24 |                                                                                    |
| 25 | <code>    queue&lt;State&gt; q;</code>                                             |
| 26 | <code>    vector&lt;vector&lt;bool&gt;&gt; visited(R, vector&lt;bool&gt;(C,</code> |
| 27 | <code>false));</code>                                                              |

```

28     int dr[] = {-1, 1, 0, 0};
29     int dc[] = {0, 0, -1, 1};
30
31     q.push({SR, SC, 0});
32     visited[SR][SC] = true;
33
34     int shortestPath = -1;
35
36     while (!q.empty()) {
37         State curr = q.front();
38         q.pop();
39
40         if (curr.r == FR && curr.c == FC) {
41             shortestPath = curr.dist;
42             break;
43         }
44
45         for (int i = 0; i < 4; i++) {
46             int nr = curr.r + dr[i];
47             int nc = curr.c + dc[i];
48
49             if (nr >= 0 && nr < R && nc >= 0 && nc
50 < C) {
51                 if (grid[nr][nc] == 0 &&
52 !visited[nr][nc]) {
53                     visited[nr][nc] = true;
54                     q.push({nr, nc, curr.dist +
55 1});
56                 }
57             }
58         }
59     }

```

|    |                               |
|----|-------------------------------|
| 55 | }                             |
| 56 | }                             |
| 57 |                               |
| 58 | cout << shortestPath << "\n"; |
| 59 |                               |
| 60 | return 0;                     |
| 61 | }                             |

## B. Output Program

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> g++ soal3.cpp -o program
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> ./program
8
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7
9
16
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code>

```

Gambar 3. Screenshot Hasil Jawaban Soal 3

## C. Pembahasan

Kode ini mensimulasikan pencarian rute pada sebuah peta atau labirin, di mana pencarian dilakukan secara melebar lapis demi lapis untuk menemukan jalur paling efisien dari titik awal ke titik tujuan. Tujuan utama dari program ini adalah untuk menemukan jalur terpendek (*shortest path*) dari koordinat awal ( $SR$ ,  $SC$ ) menuju koordinat akhir ( $FR$ ,  $FC$ ) di dalam sebuah matriks berukuran  $R \times C$ . Program akan mengevaluasi setiap langkah yang memungkinkan, menghindari rintangan, dan menghindari jalan buntu atau perulangan rute, lalu mencetak jumlah langkah minimum yang diperlukan. Jika tujuan tidak dapat dicapai, program akan mengembalikan nilai  $-1$ .

Kompleksitas waktu keseluruhan dari program ini adalah  $O(R \times C)$  di mana  $R$  adalah jumlah baris dan  $C$  adalah jumlah kolom pada matriks/grid. Secara teori, algoritma *Breadth-First Search* (BFS) pada graf umum memiliki kompleksitas waktu  $O(V + E)$ , di mana  $V$  adalah jumlah simpul (*vertex*) dan  $E$  adalah jumlah sisi (*edges*). Namun, karena program ini diimplementasikan pada sebuah *Grid 2* Dimensi, analisisnya dapat diturunkan secara lebih spesifik berdasarkan dimensi matriks.

Blok kode pertama berfungsi untuk mempersiapkan lingkungan kerja dengan memanggil Pustaka `<iostream>`, `<vector>`, dan `<queue>`. Pustaka `<queue>` ditambahkan karena algoritma BFS membutuhkan antrean dengan prinsip *First-In First-Out* (FIFO) untuk memproses titik-titik secara berurutan. Selain itu, program juga mendefinisikan sebuah tipe data kustom bernama `State` menggunakan `struct`. Struktur ini bertugas untuk membungkus tiga informasi penting menjadi satu kesatuan: `r` (baris saat ini), `c` (kolom saat ini), dan `dist` (jarak atau jumlah langkah yang telah ditempuh dari titik awal). Karena blok ini hanya berupa deklarasi statis di awal program, proses ini dieksekusi dalam waktu konstan atau  $O(1)$ .

Blok kode kedua bertugas untuk membentuk ruang kerja utama berupa peta atau labirin. Program akan membaca dimensi peta, yakni jumlah baris ( $R$ ) dan kolom ( $C$ ) dengan proteksi pengaman untuk menghentikan program jika input tidak valid. Setelah dimensi diketahui, program membuat sebuah matriks 2 dimensi menggunakan `vector` dan melakukan perulangan bersarang (*nested loop*) untuk membaca status setiap sel pada peta. Pada logika grid ini, nilai yang diinput (0 dan 1) akan disimpan sebagai rujukan jalur yang bisa dilewati nantinya. Karena program harus membaca dan menyimpan input untuk setiap sel matriks satu per satu, tahapan ini memiliki kompleksitas waktu  $O(R \times C)$ .

Blok kode ketiga berfungsi sebagai tahap persiapan (*setup*) sebelum pencarian rute dimulai. Program membaca koordinat awal ( $SR$ ,  $SC$ ) dan tujuan

(FR, FC). Selanjutnya, program menyiapkan dua komponen krusial BFS, yaitu sebuah antrean `q` untuk menjadwalkan titik mana yang akan dieksplorasi dan matriks *boolean* `visited` untuk mencatat titik mana saja yang sudah pernah dikunjungi agar program tidak terjebak dalam perulangan (*infinite loop*). Blok ini juga mendeklarasikan array arah navigasi `dr` (perubahan baris) dan `dc` (perubahan kolom) yang mewakili pergerakan ke 4 arah mata angin (atas, bawah, kiri, kanan). Sebagai langkah inisialisasi awal, titik asal dimasukkan ke dalam antrean dengan jarak awal 0, statusnya ditandai sebagai "sudah dikunjungi", dan variabel `shortestPath` disetel ke -1 sebagai asumsi dasar jika rute tujuan gagal ditemukan. Operasi dasar antrean memakan waktu  $O(1)$ . Namun, pembuatan matriks *boolean* `visited` memerlukan alokasi memori yang seukuran dengan grid utama, sehingga tahapan ini juga menyumbang kompleksitas  $O(R \times C)$ .

Blok kode keempat berfungsi sebagai mesin pemroses utama algoritma BFS. Selama antrean masih memiliki titik untuk dicek (`!q.empty()`), program akan mengambil titik terdepan (`curr`) lalu mengeluarkannya dari antrean. Program langsung melakukan verifikasi apakah titik saat ini adalah titik tujuan (FR, FC). Jika ya, pencarian langsung dihentikan (`break`) dan jarak saat ini dicatat sebagai `shortestPath`. Karena sifat dasar BFS menyebar secara merata, titik pertama yang mencapai tujuan sudah dipastikan merupakan rute terpendek. Jika belum sampai tujuan, program akan menggunakan array arah (`dr`, `dc`) untuk melihat 4 sel tetangga di sekitarnya. Untuk setiap sel tetangga, program melakukan validasi ketat, yaitu sel tersebut harus berada di dalam batas ukuran matriks, bernilai 0 (bisa dilewati/bukan dinding) dan belum pernah dikunjungi sebelumnya. Jika lolos validasi, sel tetangga tersebut ditandai sudah dikunjungi dan dimasukkan ke dalam antrean dengan nilai jarak yang ditambah satu dari jarak sebelumnya (`curr.dist + 1`).

Proses memasukkan (`push`) dan mengeluarkan (`pop`) sel dari antrean memakan waktu  $O(1)$ . Pengecekan 4 arah tetangga juga merupakan operasi konstan

$O(1)$ . Karena tidak ada sel yang diproses berulang kali, maka pada skenario terburuk, program hanya akan menelusuri seluruh luas matriks secara linier. Dengan demikian, eksekusi inti algoritma ini berjalan dengan kompleksitas  $O(R \times C)$ .

Blok kode terakhir bertugas untuk mencetak hasil komputasi ke layar. Jika algoritma BFS berhasil mencapai tujuan, variabel `shortestPath` akan mencetak jumlah langkah minimum yang dibutuhkan. Namun, jika seluruh titik yang memungkinkan sudah dieksplorasi dan antrian kehabisan elemen tanpa pernah menyentuh titik tujuan karena terhalang rintangan atau buntu, maka program akan mencetak angka `-1` sesuai nilai awal deklarasinya. Kode diakhiri dengan `return 0;` yang menandakan program telah selesai dieksekusi dengan sempurna. Operasi mencetak variabel ke layar berjalan sangat cepat dengan kompleksitas  $O(1)$ .

Jika seluruh tahapan komputasi diakumulasikan, mulai dari pembacaan data  $O(R \times C)$ , pembuatan matriks validasi  $O(R \times C)$ , hingga eksekusi inti BFS  $O(R \times C)$ , maka nilai konstanta kelipatan dapat diabaikan secara asimtotik. Oleh karena itu, dapat disimpulkan bahwa Kompleksitas Waktu Keseluruhan dari program ini adalah  $O(R \times C)$ . Algoritma ini terbukti sangat efisien karena waktu komputasinya berjalan secara proporsional dan linier mengikuti penambahan ukuran baris dan kolom pada grid yang diberikan.

## SOAL 4

Diberikan sebuah labirin berukuran  $R \times C$  dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

| Input                                                                                        | Output |
|----------------------------------------------------------------------------------------------|--------|
| 5 6<br>0 0 0 1 0 0<br>0 1 0 0 0 1<br>0 1 0 1 0 0<br>0 0 0 1 1 0<br>1 1 0 0 0 0<br>0 0<br>4 5 | 4      |

### A. Source Code

Tabel 4. Source Code Soal 4

|   |                                        |
|---|----------------------------------------|
| 1 | <code>#include &lt;iostream&gt;</code> |
| 2 | <code>#include &lt;vector&gt;</code>   |
| 3 |                                        |
| 4 | <code>using namespace std;</code>      |
| 5 |                                        |

```

6   int dr[] = {-1, 1, 0, 0};
7   int dc[] = {0, 0, -1, 1};
8
9   int R, C;
10  int FR, FC;
11  int totalPaths = 0;
12
13  void solveDFS(int r, int c, vector<vector<int>>&
grid) {
14      if (r == FR && c == FC) {
15          totalPaths++;
16          return;
17      }
18
19      grid[r][c] = -1;
20
21      for (int i = 0; i < 4; i++) {
22          int nr = r + dr[i];
23          int nc = c + dc[i];
24
25          if (nr >= 0 && nr < R && nc >= 0 && nc < C)
{
26              if (grid[nr][nc] == 0) {
27                  solveDFS(nr, nc, grid);
28              }
29          }
30      }
31
32      grid[r][c] = 0;
33  }

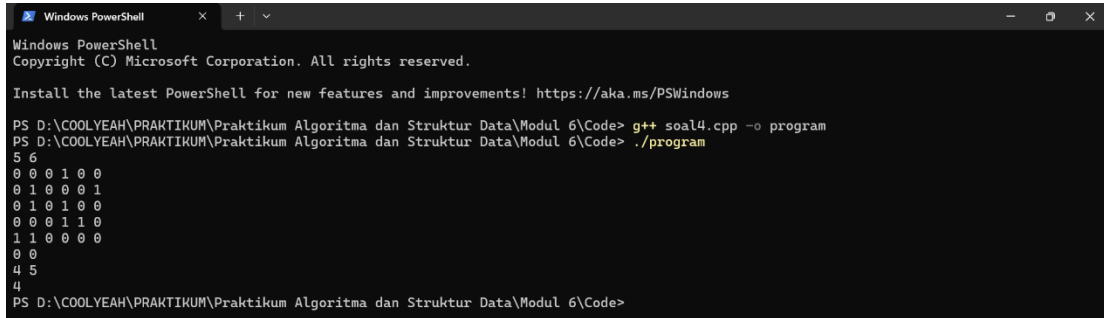
```

```

34
35 int main() {
36     if (!(cin >> R >> C)) return 0;
37
38     vector<vector<int>> grid(R, vector<int>(C));
39
40     for (int i = 0; i < R; i++) {
41         for (int j = 0; j < C; j++) {
42             cin >> grid[i][j];
43         }
44     }
45
46     int SR, SC;
47     cin >> SR >> SC >> FR >> FC;
48
49     solveDFS(SR, SC, grid);
50
51     cout << totalPaths << "\n";
52
53     return 0;
54 }

```

## B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> g++ soal4.cpp -o program
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> ./program
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code>
```

Gambar 4. Screenshot Hasil Jawaban Soal 4

## C. Pembahasan

Kode ini adalah implementasi dari algoritma pencarian Depth-First Search (DFS) yang dipadukan dengan teknik Backtracking (Runut Balik) pada matriks dua dimensi. Algoritma DFS menelusuri sebuah jalur sedalam mungkin hingga mencapai tujuan atau menemui jalan buntu, sebelum akhirnya mundur (*backtrack*) untuk mengeksplorasi cabang jalur lainnya. Program ini akan mencari dan menghitung total seluruh jalur yang memungkinkan dari titik awal (SR, SC) menuju titik akhir (FR, FC), dan akan mengeksplorasi setiap kombinasi rute yang ada tanpa menabrak rintangan dan tanpa melewati sel yang sama lebih dari satu kali dalam satu perjalanan rute. Total variasi jalur yang berhasil mencapai tujuan kemudian dicetak ke layar.

Blok kode pertama berfungsi sebagai persiapan lingkungan kerja dengan pustaka yang diperlukan. Baris ini juga mendeklarasikan beberapa variabel global agar dapat diakses langsung oleh fungsi rekursif nantinya. Variabel array `dr` dan `dc` berfungsi sebagai vektor arah untuk memandu pergerakan ke 4 sisi. Variabel `R`, `C`, `FR`, dan `FC` menyimpan dimensi grid dan titik tujuan, sedangkan variabel `totalPaths` diinisialisasi dengan angka 0 sebagai wadah (*counter*) untuk menghitung rute yang berhasil.

Blok kode kedua bertugas sebagai mesin utama dari program. Fungsi `solveDFS` dieksekusi secara rekursif (memanggil dirinya sendiri) untuk menelusuri jalur. Pada setiap pemanggilan, program akan mengecek apakah posisi saat ini (`r`, `c`) adalah titik tujuan (`FR`, `FC`). Jika ya, `totalPaths` bertambah 1 dan pencarian di cabang tersebut dihentikan (`return`). Sel yang sedang diinjak diubah nilainya menjadi `-1`. Ini adalah bentuk validasi agar sel ini tidak dilewati dua kali dalam satu jalur yang sama, sehingga mencegah perulangan tak hingga (*infinite loop*). Program mencoba melangkah ke 4 arah menggunakan perulangan. Jika sel tetangga berada di dalam batas grid dan bernilai 0 (jalan bebas), maka fungsi `solveDFS` dipanggil kembali untuk sel tersebut. Setelah seluruh 4 arah selesai dieksplorasi, nilai sel dikembalikan menjadi 0. Proses pembatalan status (*unmarking*) inilah yang disebut *backtracking*. Tujuannya agar sel tersebut "terbuka" kembali dan bisa digunakan oleh rute kombinasi lain yang sedang dicari.

Blok terakhir berfungsi untuk mengatur alur data. Program membaca dimensi baris dan kolom dengan pengaman validasi, lalu membentuk matriks dua dimensi (`grid`) untuk menyimpan peta labirin melalui *nested loop*. Setelah itu, program membaca titik koordinat awal (`SR`, `SC`) dan akhir (`FR`, `FC`). Pemanggilan fungsi `solveDFS` pertama kali dilakukan dari titik awal ini. Setelah proses komputasi rekursif selesai mendata seluruh jalur yang ada, fungsi `main` akan mencetak hasil akumulasi `totalPaths` ke layar dan menutup program dengan instruksi `return 0;`.

Pencarian total jalur menggunakan DFS dengan mekanisme *Backtracking* memiliki sifat pertumbuhan eksponensial. Berbeda dengan pencarian jalur terpendek (seperti BFS yang bernilai linier), program ini secara paksa menelusuri *semua* kemungkinan kombinasi percabangan. Pada setiap sel (titik persimpangan), program dapat melangkah ke maksimal 4 arah. Namun, karena program tidak bisa melangkah kembali ke kotak sebelumnya (karena sudah ditandai dengan `-1`), cabang efektif yang dapat dipilih pada setiap langkah adalah maksimal 3 arah.

Dalam skenario terburuk (*worst-case scenario*) seperti pada grid kosong tanpa rintangan, panjang satu jalur maksimal bisa menyentuh seluruh luas matriks, yaitu sejauh  $R \times C$  Langkah. Karena pada setiap langkah memiliki 3 hingga 4 pilihan cabang dan kedalaman langkah maksimal adalah total luas grid, maka kompleksitas waktu keseluruhan secara teoritis berada pada kisaran  $O(4^{R \times C})$  atau dapat diestimasi lebih ketat menjadi batas atas  $O(3^{R \times C})$ . Kompleksitas eksponensial ini menandakan bahwa algoritma *DFS + Backtracking* sangat efektif untuk menghitung seluruh kemungkinan jalur, namun hanya optimal digunakan pada grid berukuran kecil. Jika ukuran baris (R) dan kolom (C) sangat besar, program akan mengalami lonjakan waktu eksekusi yang masif (*Time Limit Exceeded*).

## SOAL 5

Diberikan  $N$  bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red-Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTTree terbentuk, diberikan  $Q$  query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

| Input                | Output                             |
|----------------------|------------------------------------|
| 7                    | [Preorder] : 50 30 20 40 70 60 80  |
| 50 30 70 20 40 60 80 | [Inorder] : 20 30 40 50 60 70 80   |
| 4                    | [Postorder] : 20 40 30 60 80 70 50 |
| PREORDER             | [Preorder] : 50 30 20 40 70 60 80  |
| INORDER              | [Inorder] : 20 30 40 50 60 70 80   |
| POSTORDER            | [Postorder] : 20 40 30 60 80 70 50 |
| ALL                  |                                    |

### A. Source Code

Tabel 5. Source Code Soal 5

|   |                     |
|---|---------------------|
| 1 | #include <iostream> |
| 2 | #include <string>   |
| 3 | #include <vector>   |

```

4  #include "RedBlackTree.h"
5
6  using namespace std;
7
8  void traversePreorder(const RedBlackTree::Node*
node) {
9      if (node->isNil) return;
10     cout << " " << node->key;
11     traversePreorder(node->left);
12     traversePreorder(node->right);
13 }
14
15 void traverseInorder(const RedBlackTree::Node*
node) {
16     if (node->isNil) return;
17     traverseInorder(node->left);
18     cout << " " << node->key;
19     traverseInorder(node->right);
20 }
21
22 void traversePostorder(const RedBlackTree::Node*
node) {
23     if (node->isNil) return;
24     traversePostorder(node->left);
25     traversePostorder(node->right);
26     cout << " " << node->key;
27 }
28
29 int main() {
30     int N;

```

```

31     if (!(cin >> N)) return 0;
32
33     RedBlackTree rbt;
34
35     for (int i = 0; i < N; i++) {
36         int val;
37         cin >> val;
38         if (!rbt.contains(val)) {
39             rbt.insert(val);
40         }
41     }
42
43     int Q = 0;
44     vector<string> queryList;
45
46     if (cin >> Q) {
47         for (int i = 0; i < Q; i++) {
48             string query;
49             cin >> query;
50             queryList.push_back(query);
51         }
52     }
53
54     if (rbt.empty()) {
55         cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
56         return 0;
57     }
58
59     for (int i = 0; i < Q; i++) {

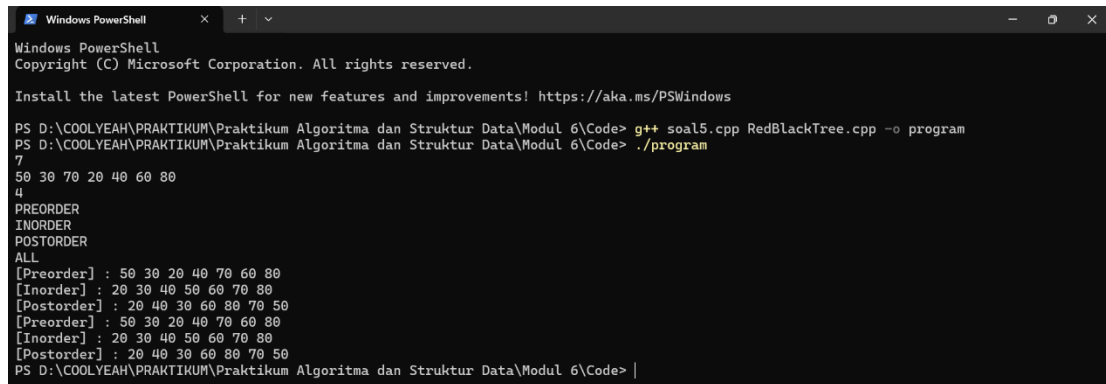
```

```

60         string query = queryList[i];
61
62         if (query == "PREORDER" || query == "ALL")
63         {
64             cout << "[Preorder] :";
65             traversePreorder(rbt.root());
66             cout << "\n";
67         }
68         if (query == "INORDER" || query == "ALL") {
69             cout << "[Inorder] :";
70             traverseInorder(rbt.root());
71             cout << "\n";
72         }
73         if (query == "POSTORDER" || query == "ALL")
74         {
75             cout << "[Postorder] :";
76             traversePostorder(rbt.root());
77             cout << "\n";
78         }
79     }
80     return 0;
81 }

```

## B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> g++ soal5.cpp RedBlackTree.cpp -o program
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> ./program
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS D:\COOLYEAH\PRAKTIKUM\Praktikum Algoritma dan Struktur Data\Modul 6\Code> |
```

Gambar 5. Screenshot Hasil Jawaban Soal 5

## C. Pembahasan

Kode ini bertujuan untuk memasukkan sekumpulan angka (node) ke dalam *Red-Black Tree* sehingga tidak ada data duplikat di dalamnya. Setelah pohon terbentuk, program akan menerima sejumlah instruksi (kueri) dari pengguna untuk menampilkan isi pohon tersebut menggunakan tiga metode penelusuran (*Tree Traversal*) standar, yaitu Preorder, Inorder, Postorder, dan gabungan ketiganya (ALL).

Blok kode pertama berfungsi untuk mempersiapkan lingkungan kerja. `<iostream>` digunakan untuk menangani *input/output*, `<string>` untuk memproses teks kueri, dan `<vector>` untuk menyimpan daftar perintah secara dinamis. Perintah `#include "RedBlackTree.h"` untuk memanggil definisi struktur dan algoritma mandiri dari *Red-Black Tree* yang dibuat di luar program utama.

Blok kode kedua memiliki tiga fungsi yang bertugas untuk menelusuri dan mencetak isi dari *Red-Black Tree* secara rekursif. Setiap fungsi memiliki *Base Case* `if (node->isNil) return;`, yang menghentikan rekursi jika mencapai *sentinel node* atau daun kosong (Nil) yang merupakan ciri khas dari struktur RBT. `traversePreorder` berfungsi untuk mengunjungi simpul akar

(Root) terlebih dahulu, kemudian ke anak kiri (Left), lalu ke anak kanan (Right), `traverseInorder` untuk mengunjungi anak kiri (Left), lalu mencetak akar (Root), lalu ke anak kanan (Right). Pada pohon pencarian biner, *Inorder* akan selalu menghasilkan urutan data yang telah disortir dari kecil ke besar, dan `traversePostorder` untuk mengunjungi anak kiri (Left), lalu anak kanan (Right), dan terakhir baru memproses/mencetak akar (Root).

Blok kode ketiga berfungsi untuk membaca jumlah data *N* dan menginisialisasi objek `rbt` dari kelas `RedBlackTree`. Program kemudian menggunakan perulangan untuk membaca nilai angka. Terdapat mekanisme penyaringan menggunakan `!rbt.contains(val)`; program memvalidasi apakah angka tersebut sudah ada di dalam pohon. Jika belum ada (bernilai *false*), barulah angka tersebut dimasukkan menggunakan `rbt.insert(val)`. Hal ini menjamin integritas data yang bersifat unik (*distinct*).

Blok kode keempat bertugas untuk pembacaan dan penyimpanan kueri. Setelah pohon berhasil dibangun, program akan meminta input jumlah kueri yang disimpan dalam variabel *Q*. Program lalu menyiapkan *dynamic array* bernama `queryList` untuk menampung perintah teks tersebut. Melalui *looping*, setiap instruksi teks (seperti "PREORDER" atau "ALL") dimasukkan (*push\_back*) secara berurutan ke dalam memori *vector* untuk diproses di tahap akhir.

Blok kode terakhir adalah eksekutor instruksi. Pertama-tama, program mengecek apakah pohon tidak memiliki data sama sekali menggunakan `rbt.empty()`. Jika kosong, program mencetak pesan peringatan dan langsung berhenti. Jika ada data, program akan melakukan perulangan sebanyak kueri yang ada, lalu mengeksekusi pencetakan struktur menggunakan fungsi `traverse` yang sesuai dengan teks kuerinya. Jika kuerinya adalah "ALL", maka ketiga metode pencetakan tersebut akan dijalankan secara beruntun.

Program ini memiliki efisiensi yang sangat baik dan konsisten karena memanfaatkan *Red-Black Tree*. Analisis kompleksitas waktunya dapat dibagi

menjadi beberapa tahapan, yaitu tahap penyisipan data (*Insertion*), tahap pembacaan kueri, dan tahap penelusuran (*Traversal*). Pada tahap *Insertion*, *Red-Black Tree* secara otomatis menyeimbangkan dirinya sehingga tinggi pohon selalu berada pada batasan logaritmik, yaitu  $O(\log U)$ , di mana  $U$  adalah jumlah node unik. Dalam perulangan  $N$ , operasi `contains()` dan `insert()` masing-masing memakan waktu  $O(\log U)$ . Kompleksitas tahap ini adalah  $O(N \log U)$ . Pada kasus terburuk (*worst case*), yang jauh lebih aman dari *Binary Search Tree* biasa yang bisa memburuk hingga  $O(N^2)$ .

Pada tahap kueri, program menyimpan instruksi ke dalam *vector* sebanyak  $Q$  kali memakan waktu konstan tiap operasinya. Kompleksitas pada tahap ini adalah  $O(Q)$ . Pada tahap *Traversal*, fungsi rekursif `Preorder`, `Inorder`, maupun `Postorder` dirancang untuk mengunjungi setiap node tepat satu kali. Oleh karena itu, satu kali pencetakan penuh memakan waktu linier terhadap jumlah data unik di dalam pohon, yaitu  $O(U)$ . Karena proses ini dijalankan berdasarkan jumlah kueri  $Q$ , maka kompleksitas maksimalnya adalah perkalian antara jumlah instruksi dan jumlah data. Kompleksitas pada tahap ini adalah  $O(Q \times U)$ .

Kompleksitas waktu keseluruhan (*Overall Time Complexity*) dari program ini adalah  $O(N \log U + Q \times U)$ , di mana  $N$  adalah jumlah data awal,  $U$  adalah jumlah data unik, dan  $Q$  adalah jumlah kueri yang diinputkan. Program ini sangat ideal dan efisien meskipun diuji dengan rentang kumpulan data yang besar berkat stabilitas *self-balancing* dari *Red-Black Tree*.